Step 2 — A Supervisor Routes Between Them — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and the boundary
- Hint 2 — The plan, and async nodes

Hint 1 — The idea, and the boundary

Project 4's `Command` Supervisor, over Step 1's specialists. The boundary that matters: **the Supervisor only routes** — it returns `Command(goto="notes"/"web"/"writer")` and reads results from shared state; it never touches a `ClientSession`. Add:

- `src/state.py` — `TeamState` (task, notes_findings, web_findings, web_unavailable, report, routing_log).
- `src/agents/supervisor.py` — `decide_next(state)` (route by what's still needed) and `supervisor(state)` returning a `Command`.
- `src/graph.py` — `START -> supervisor`; specialist nodes are **async** and return `Command(goto="supervisor", update=...)`.

Hint 2 — The plan, and async nodes

```
decide_next(state):
    if needs_notes(task) and notes_findings == "":   return "notes"
    if needs_web(task) and web_findings=="" and not web_unavailable:
        return "web"
    return "writer"
supervisor(state):
    Command(goto=decide_next(state), update={routing_log+[target]})
notes_node/web_node: async; await run_*_agent(...)
    -> Command(goto="supervisor", update=...)
run the graph with: await app.ainvoke(start_state(task))
```

The one thing people get wrong: a specialist node must be `async def` and the graph run with `.ainvoke()`. Don't wrap the async specialist in `asyncio.run()` inside a sync node — it fights LangGraph's own loop and hangs. The whole file set is in the Solution.